

## Entregable 2 - Concurrencia

Fecha límite de entrega: 29/09 12:00hs por tarea de Moodle.

Grupo de 3 personas como máximo. Avisar al docente en clase si cambia la conformación del grupo.

Fecha de presentación en clase: 28/09 - 29/09 - Deben estar presentes todos los integrantes del grupo.

### Requerimientos funcionales

Desarrolle un programa en Java que simule el funcionamiento de un restaurante donde múltiples clientes ingresan al local a comer, mozos atienden a los clientes, cocineros cocinan la comida y cajeros cobran.

Se debe desarrollar el programa utilizando las herramientas vistas en clases sobre programación concurrente. El sistema funciona automáticamente y se mostrará en pantalla el estado de cada cambio.

La simulación debe ejecutarse durante un tiempo  $T$ . Transcurrido este tiempo se simula el cierre del restaurante debiendo finalizar todos los hilos y liberar todos los recursos.

Vamos a utilizar algunas constantes configuradas en el sistema. Estas constantes son:

- $T$  = tiempo de la simulación.
- $M$  = cantidad de mesas.
- $P$  = personas por mesa.
- $Z$  = cantidad de mozos.
- $C$  = cantidad de cocineros.
- $Y$  = cantidad de cajas.
- $TP_{min}$ ,  $TP_{max}$  = tiempo mínimo y máximo que puede llegar un cliente al restaurante.
- $TM_{min}$ ,  $TM_{max}$  = tiempo mínimo y máximo que demoran los clientes en seleccionar el menú.
- $TQ_{min}$ ,  $TQ_{max}$  = tiempo mínimo y máximo que demoran los clientes en comer.
- $TZ_{min}$ ,  $TZ_{max}$  = tiempo mínimo y máximo que demoran los mozos en anotar el pedido.
- $TR_{min}$ ,  $TR_{max}$  = tiempo mínimo y máximo que demoran los mozos en retirar los platos de la cocina y servirlos en la mesa de los clientes.
- $TL_{min}$ ,  $TL_{max}$  = tiempo mínimo y máximo que demoran los mozos en levantar los platos y limpiar la mesa.
- $TC_{min}$ ,  $TC_{max}$  = tiempo mínimo y máximo que demora un menú en cocinarse. Existe una lista de menús donde cada uno tiene un rango de tiempos distintos.
- $TY_{min}$ ,  $TY_{max}$  = tiempo mínimo y máximo que demoran los cajeros en cobrar a los clientes.

Tendremos los siguientes roles en el restaurante, y cada uno de ellos debe ejecutarse de forma concurrente.

### Clientes

Pueden ingresar al restaurante  $P * M$  clientes. Si llegan más clientes al restaurante deben esperar afuera.

Las acciones que realizarán los clientes son las siguientes:

- Ingresan al restaurante.
- Se agrupan una cantidad  $P$  de clientes para poder sentarse en una mesa libre. Los  $P$  clientes se sientan en la misma mesa. Si son menos de  $P$  clientes se quedan esperando.
- Seleccionan qué comer del menú. Luego de que los  $P$  clientes seleccionan el menú llaman a un mozo. Demora un tiempo aleatorio entre  $TM_{min}$  y  $TM_{max}$  seleccionar el menú. El menú se selecciona de forma aleatorio entre una lista de menús disponibles.
- Esperan que venga un mozo.
- Cuando el mozo llega hacen el pedido.
- Esperan que el mozo traiga la comida para los  $P$  clientes.
- Luego que todos los clientes de la mesa están servidos comen. Demoran un tiempo aleatorio entre  $TQ_{min}$  y  $TQ_{max}$  en comer.
- Cada uno de forma individual, a medida que terminan de comer, van a la o las cajas a pagar.
- Se retiran de forma individual del restaurante. Cuando se van los  $P$  clientes del restaurante la mesa queda libre.

### Mozos

Existe una cantidad  $Z$  de mozos, fija. Los mozos no tienen asignadas mesas específicas. Realizan cualquiera de las siguientes acciones sin un orden específico:

- Toman el pedido de los clientes cuando lo llaman de alguna mesa y le pasan el pedido a la cocina. El mozo demora un tiempo aleatorio entre  $TZ_{min}$  y  $TX_{max}$  anotar el pedido e ir a la cocina para pasarle el pedido a los cocineros.
- Recogen los platos prontos de la cocina para una mesa, para los  $P$  clientes de la mesa, y se los lleva a los clientes de la mesa correspondiente. El mozo demora un tiempo aleatorio entre  $TR_{min}$  y  $TR_{max}$  en agarrar los platos de la cocina y servirlos en la mesa de los clientes.
- Levanta los platos y limpia la mesa que quedó sin clientes. Una vez que finaliza la mesa queda marcada como libre. El mozo demora un tiempo aleatorio entre  $TL_{min}$  y  $TL_{max}$  en levantar los platos y limpiar la mesa.
- Si los clientes no llaman para que tomen sus pedidos o no hay platos para retirar de la cocina se quedan esperando.

## Cocineros

Existe una cantidad  $C$  de cocineros, fija. Los cocineros cocinan los menús de los pedidos por orden de llegada de cada pedido.

- Un cocinero libre toma del primer pedido de la cola algún menú sin cocinar. Si no hay pedidos se quedan esperando.
- Cocina el menú seleccionado. Demora un tiempo aleatorio entre  $TC_{min}$  y  $TC_{max}$  del menú específico en cocinar el menú.
- Deja el plato pronto para retirar. Si todos los platos del pedido están prontos avisa a los mozos que pueden venir a retirarlos.
- Si no tienen pedidos para cocinar se quedan esperando.

## Cajeros

Existen una cantidad  $Y$  de cajeros. Los clientes hacen cola para pagar. Existe una única cola de clientes y los cajeros van cobrando al primero de la lista:

- El cobro del menú a cada cliente. Demora un tiempo aleatorio entre  $TY_{min}$  y  $TY_{max}$  en cobrarle al cliente.
- Si no existen clientes en la cola para pagar se quedan esperando.

## Terminación de la simulación

Transcurrido el tiempo  $T$  que indica la finalización de la simulación debemos:

- Cerrar las puertas del restaurante impidiendo que entren más clientes.
- Los clientes que todavía no realizaron sus pedidos deberán retirarse.
- Los clientes que ya realizaron sus pedidos continúan con los pasos normales.
- Los mozos, cocineros y cajeros continúan trabajando hasta no tener más clientes en el restaurante.

## Display en pantalla

Se deberá proveer un mecanismo para actualizar la pantalla. Para implementar esto, sugerimos un hilo encargado de esta tarea, que reciba las notificaciones de cada cambio, en una cola y los vaya procesando en orden.

Al finalizar el tiempo  $T$ , el display deberá continuar mostrando todos los cambios realizados y cuando no se tenga ningún cambio más en la cola se finaliza.

Pedimos que por cada cambio se muestre en pantalla el estado del restaurante del momento que se mandó la notificación, con todos los actores (clientes, mozos, cocineros y cajeros).

## Mejoras en la solución

Cumpliendo con los requisitos pedidos, tómense la libertad de agregar todas las herramientas y mejoras que consideren a la simulación, tanto para hacerlo más eficiente, como más interesante y desafiante.

## Requerimientos técnicos

1. La solución debe ser modular y seguir las mejores prácticas de concurrencia en Java, incluyendo la gestión de excepciones y el uso de estructuras de datos seguras para hilos.
2. Debe utilizar las siguientes herramientas:
  - a. Al menos dos herramientas distintas de exclusión mutua para compartir de forma segura la memoria compartida (synchronized, clases atómicas, ReentrantLock, ReentrantReadWriteLock, semáforos, etc)
  - b. Alguna de las herramienta de sincronización con barrera (CountDownLatch, CyclicBarrier, Phaser)
  - c. Alguna herramienta de pasaje de mensajes (BlockingQueue, ConcurrentLinkedQueue, etc.)
  - d. Pool de hilos (Executors)
  - e. Uso de wait / notify / notifyAll
3. Utilice las herramientas obligatorias que le pedimos y cualquier otra que permita asegurar la correcta sincronización del juego, evitar cualquier tipo de bloqueo, como por ejemplo deadlocks, y asegurar que los datos compartidos se mantengan visibles para todos (sin uso de caché).
4. Utilice al menos un patrón de diseño concurrente y justifique su elección en la documentación.
5. Debe guardarse un log en un archivo de texto donde se guarden todas las acciones realizadas y los cambios que ocurren en la simulación.
6. La documentación PDF debe contener como mínimo:
  - a. Breve detalle de las herramientas utilizadas y justificación para la elección de las mismas (seguridad de hilos, evitación de deadlocks, etc).
  - b. Decisiones de funcionamiento del simulador, de diseño e implementación tomadas.
  - c. Breve conclusión del producto desarrollado y del aprendizaje obtenido.

## Forma de entrega

Se debe subir un zip, con el nombre Entregable2Apellido1Apellido2.zip, ejemplo (Entregable2GarciaPerez.zip) que contenga los apellidos de los integrantes del grupo.

El zip debe contener al menos:

- ☐ Código fuente
- ☐ Documentación técnica de tipo informe PDF.

## Anexo - Requerimientos extra (grupos de 3 personas)

### Requerimiento extra 1:

Cada cocinero necesita dos sartenes para cocinar. Los cocineros tienen una cantidad total  $Q$  de sartenes para utilizar. La constante  $Q$  debe ser al menos 2. Una vez que un cocinero agarra una sartén esta no puede ser usado por otro cocinero hasta que sea liberada.

Cada cocinero debe :

- Agarrar una sartén.
- Agarrar otra sartén
- Cocina el menú seleccionado.
- Libera una sartén.
- Libera otra sartén.

Implementar el código de los cocineros teniendo especial cuidado en evitar deadlock. Explique en la documentación qué hicieron para evitarlo.

### Requerimiento extra 2:

El restaurante agrega el servicio de Take Away ("Para llevar"). Se agrega un empleado dedicado a tomar estos pedidos de menús (que son los mismos de la carta). Realiza las siguientes tareas:

- Atiende a los clientes que están esperando para realizar un pedido para llevar.
- Cobra el pedido del cliente. El cliente se queda esperando.
- Le pasa el pedido, con el menú, a los cocineros.
- Cuando un pedido queda pronto va a la cocina a buscarlo y se le entrega al cliente. La prioridad la tiene la entrega de los platos listos antes que tomar nuevos pedidos y cobrar.

Cuando un pedido de Take Away llega a la cocina, tiene prioridad sobre los menús de las mesas.

Los clientes pueden ingresar al restaurante aunque esté lleno, si van a pedir para llevar. Este tipo de clientes ingresan al restaurante cada un tiempo aleatorio entre  $TW_{min}$  y  $TW_{max}$ .

Indique en la documentación qué problemas podrían surgir con este nuevo requerimiento de agregar prioridades.